Web and Data Engineering

Kapitel 3: Frontend-Technologien — HTML, JavaScript und CSS

Prof. Dr. Michael Granitzer

Ziel dieser Vorlesung

Jede Web-Anwendung, die ein Nutzer sieht und bedient, besteht aus drei Technologien: HTML für Struktur, CSS für Darstellung und JavaScript für Verhalten. Diese Vorlesung vermittelt die **Grundlagen** aller drei — nicht als Programmierkurs, sondern als **Architekturwissen** für Web Engineers.

Leitfragen

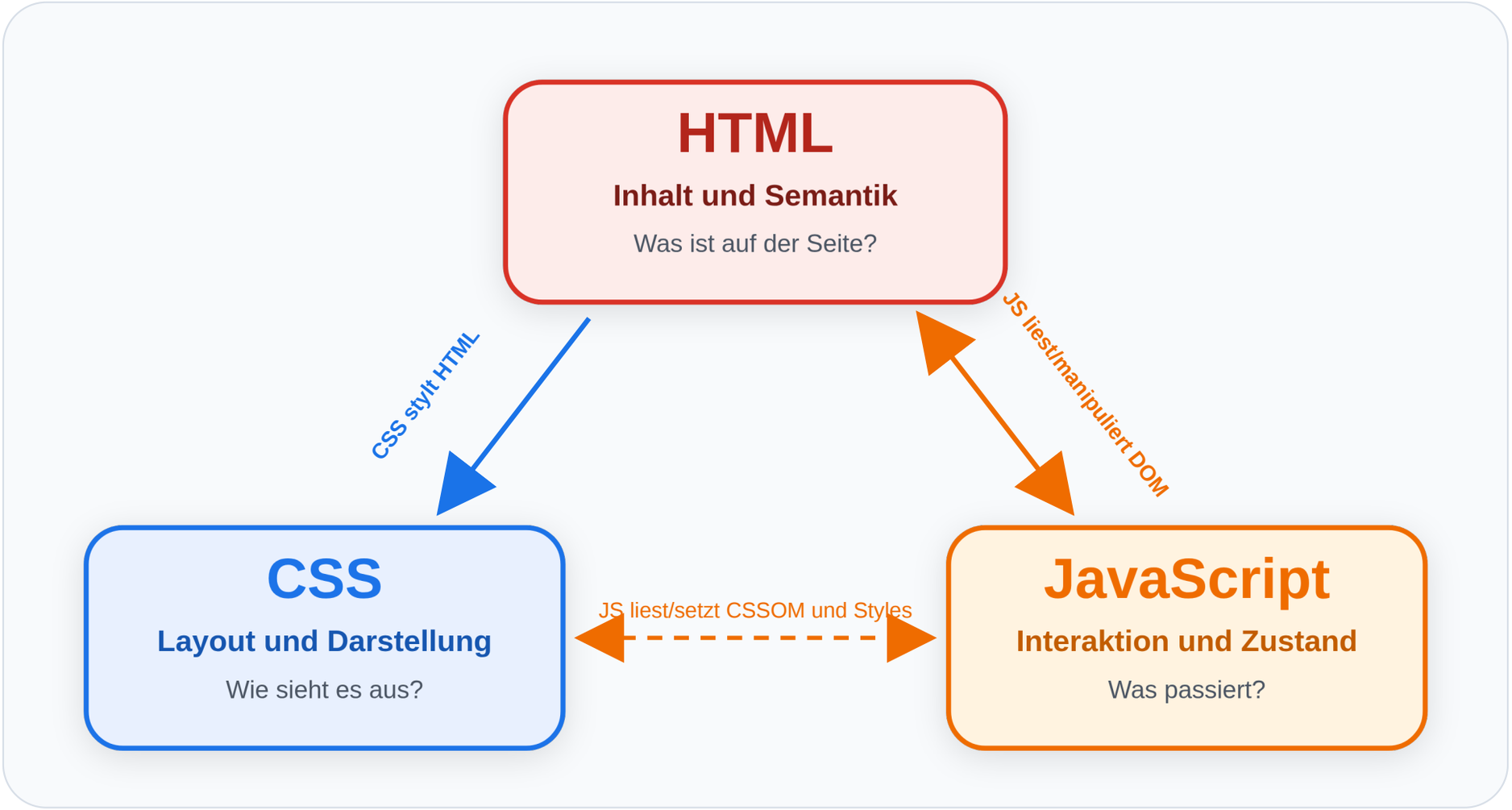
- Warum ist semantisches HTML mehr als Markup?
- Wie macht JavaScript den Browser zur Laufzeitumgebung?
- Wie erzeugt CSS aus derselben Struktur unterschiedliche Layouts?
- Wie arbeiten HTML, CSS und JavaScript im Browser zusammen?

HTML, CSS und JavaScript — Zusammenspiel im Browser

Leitfrage: Welche Aufgabe haben HTML, CSS und JavaScript jeweils im Browser — und wie greifen sie zusammen, bevor wir die drei Technologien einzeln vertiefen?

Das Technologie-Dreieck des Webs

> **Architekturentscheidung:** Inhalt gehört in HTML, Layout in CSS, Interaktion in JavaScript.


HTML
Inhalt und Semantik
Was ist auf der Seite?
CSS stylt HTML
JS liest/manipuliert DOM
CSS
Layout und Darstellung
Wie sieht es aus?
JS liest/setzt CSSOM und Styles
JavaScript
Interaktion und Zustand
Was passiert?

Hinter den Standards: W3C, WHATWG und die Standardisierung des Webs

Die drei Frontend-Technologien sind keine Produkte einer einzelnen Firma, sondern Ergebnis von Standardisierung.

- **WHATWG** (*Web Hypertext Application Technology Working Group*) ist eine von Browser-Herstellern geprägte Arbeitsgruppe und pflegt heute den **HTML Living Standard** sowie das **DOM**.
- **W3C** (*World Wide Web Consortium*) ist das breitere Standardisierungskonsortium des Webs; dort liegen u. a. **CSS** und **ARIA (Accessible Rich Internet Applications)**. (e.g. Screenreader).
- **ECMA International** standardisiert **JavaScript** unter dem Namen **ECMAScript**.

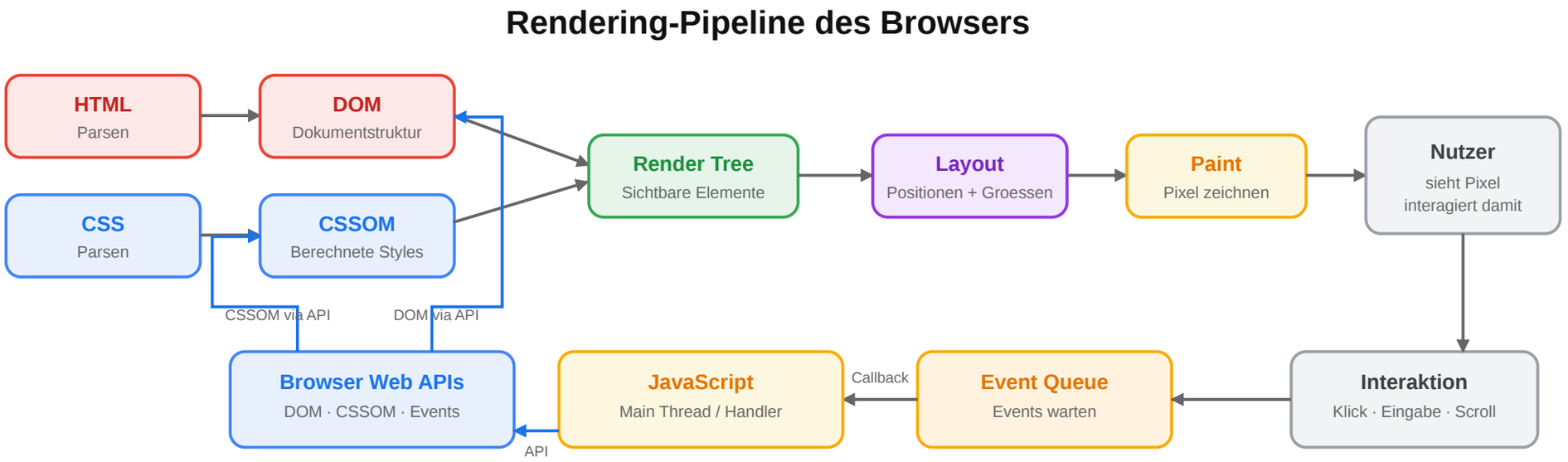
Wichtig: HTML lag früher auch beim W3C. Heute ist die WHATWG die primäre Instanz für HTML, während das W3C für andere zentrale Webstandards weiter wichtig bleibt. Browser implementieren diese Spezifikationen. Deshalb funktioniert dieselbe Seite grundsätzlich in Chrome, Firefox und Safari.

Ein Seitenaufruf in drei Schritten

1. **HTML:** Der Browser lädt das Dokument und baut daraus das **Document Object Model (DOM)** auf.
2. **CSS:** Stylesheets (Cascading Style Sheets) werden geladen. Der Browser berechnet daraus, wie der Inhalt aussieht.
3. **JavaScript:** Skripte registrieren Verhalten, reagieren auf Events und können DOM und Styles später verändern.

> **Wichtig:** HTML kommt logisch zuerst. CSS und JavaScript bauen auf dieser Struktur auf.

Die Rendering-Pipeline des Browsers



Die Pipeline arbeitet auf zwei internen Strukturen: dem **DOM** für HTML und dem **CSSOM** für CSS. Erst daraus erzeugt der Browser den Render Tree und rechnet Layout, Paint und Compositing aus.

Was passiert bei einem Klick?

1. HTML enthält z. B. einen Button und einen Warenkorb-Zähler.
2. CSS definiert, wie Button, Badge und Ladezustand aussehen.
3. JavaScript reagiert auf den Klick.
4. JavaScript ändert DOM oder Klassen.

5. Der Browser rendert die sichtbare Änderung neu.
6. Optional sendet JavaScript asynchron einen Request an den Server.

Fahrplan: vom Überblick ins Detail

Modul 1: HTML

Semantik, Struktur, Formulare, eingebautes Verhalten

Modul 2: JavaScript

Code im Browser, DOM, Events, Fetch, Alternativen

Modul 3: CSS

Cascade, Layoutmodelle, Grid, Responsive Design

Danach folgt eine **Vertiefung**: Web-APIs und moderne Frontend-Architekturen.

HTML — Struktur und Semantik

Leitfrage: Warum reicht es nicht, eine Web-Seite einfach mit `<div>`-Elementen zusammenzubauen — und was gewinnt man durch semantisches HTML?

Zwei Versionen derselben Seite

Version A

```
<div class="top">
  <div class="logo">Shop</div>
  <div class="links">
    <div><a href="/">Start</a></div>
    <div><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="m">
  <div class="l">
    <div class="t">Funkkopfhörer</div>
    <div>Kabelloser Kopfhörer mit...</div>
    <div class="price">€ 79,99</div>
  </div>
  <div class="r">
    <div>Auch interessant: ...</div>
  </div>
</div>
```

Version B

```
<header>
  <h1>Shop</h1>
  <nav>
    <a href="/">Start</a>
    <a href="/products">Produkte</a>
  </nav>
</header>
<main>
  <article>
    <h2>Funkkopfhörer</h2>
    <p>Kabelloser Kopfhörer mit...</p>
    <p class="price">€ 79,99</p>
  </article>
  <aside>Auch interessant: ...</aside>
</main>
<footer>© 2026 Shop</footer>
```

> **Aufgabe:** Beide Versionen sehen im Browser identisch aus. Was unterscheidet sie — und für wen?

Was Version B besser macht

Nutzer	Version A (`<div>`)	Version B (semantisch)
Screenreader	„div, div, div, Link, div, div..." — keine Orientierung	„Navigation mit 2 Links, Hauptinhalt: Artikel ‚Funkkopfhörer', Sidebar"
Suchmaschine	Muss raten, was Hauptinhalt ist	`<article>` + `<h2>` = klares Inhaltsranking
Entwickler	Muss CSS-Klassen lesen, um Struktur zu verstehen	Code ist selbstdokumentierend
Browser	Kein Reader-Mode, keine sinnvolle Druckansicht	Reader-Mode extrahiert `<article>`, Druckansicht setzt `<main>`

Kernprinzip

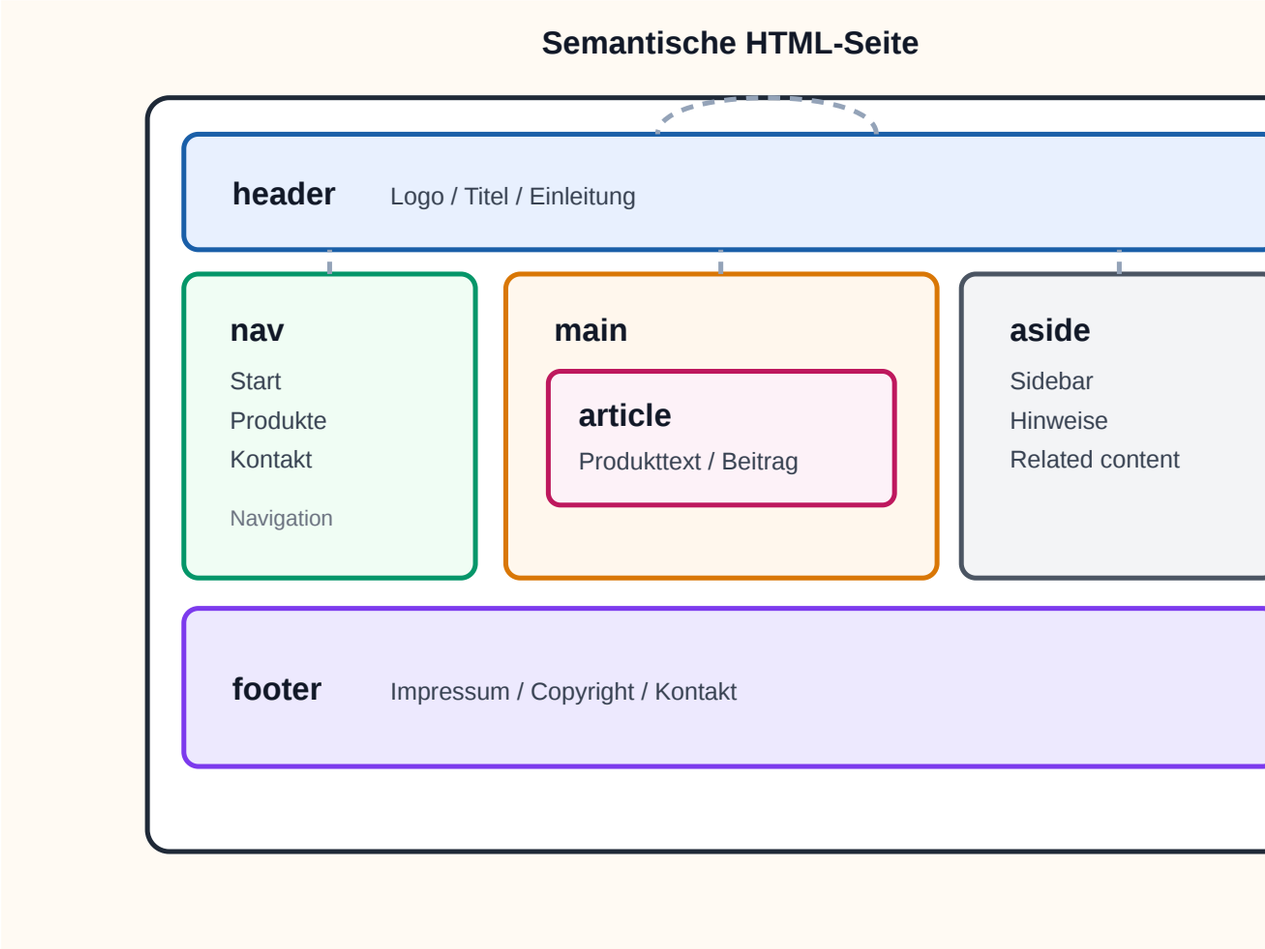
HTML trennt

- **Struktur** (was ist ein Absatz, eine Überschrift, eine Navigation?)
- von **Darstellung** (wie sieht es aus? → CSS)
- und **Verhalten** (was passiert bei Klick? → JavaScript).

Semantische Elemente in HTML5

HTML5 hat generische Container (`<div>`, `<span>`) durch bedeutungstragende Elemente ergänzt:

Element	Bedeutung
`<header>`	Kopfbereich einer Seite oder Sektion
`<nav>`	Navigationsbereich
`<main>`	Hauptinhalt der Seite (einmalig)
`<article>`	Eigenständiger Inhalt (Blogpost, Produkt)
`<section>`	Thematischer Abschnitt
`<aside>`	Ergänzender Inhalt (Sidebar, Hinweis)
`<footer>`	Fußbereich
`<figure>` / `<figcaption>`	Bild mit Beschriftung



Faustregel Wenn ein Element eine **inhaltliche Bedeutung** hat, gibt es dafür ein semantisches HTML-Element. `<div>` ist für Layout-Container ohne eigene Bedeutung.

Semantik kann mehr: Microformats und Microdata

Semantik in HTML endet nicht bei `<header>` und `<article>`. Für maschinenlesbare Zusatzinformationen gibt es auch Microformats und Microdata.

```html
<a class="h-card" href="/team/max">
  <span class="p-name">Max Mustermann</span>
  <img class="u-photo" src="/img/max.jpg" alt="">
</a>

<article itemscope itemtype="https://schema.org/Person">
  <h1 itemprop="name">Max Mustermann</h1>
  <p itemprop="jobTitle">Data Engineer</p>
</article>
```

- **Microformats** nutzen Klassen wie `h-card`, `p-name`, `u-photo`
- **Microdata** nutzt `itemscope`, `itemtype`, `itemprop`
- Beide ergänzen vorhandenes HTML statt es zu ersetzen

> Details: MDN Microformats und MDN Microdata

Wofür braucht man das?

Microformats und Microdata sind nützlich, wenn HTML nicht nur für Menschen lesbar sein soll, sondern auch für Programme, die Inhalte erkennen, zusammenfassen oder weiterverarbeiten.

Microformats

- oft für Personen, Kontakte, Termine, Social Links
- leichtgewichtig und direkt lesbar
- Beispiel: `h-card`, `h-event`, `p-name`

Microdata

- häufig für `Person`, `Product`, `Event`
- enger an Vokabulare wie `schema.org` gebunden
- Beispiel: `itemscope`, `itemtype`, `itemprop`

Typische Nutzung



Was die Nutzung bringt

- bessere maschinelle Erkennbarkeit von Inhalten
- mögliche Rich Results in Suchmaschinen
- weniger Parsing-Hacks in externen Tools
- HTML bleibt die primäre Quelle

- **Suchmaschinen:** Produkte, Personen, Organisationen, Events, Bewertungen
- **Aggregatoren und Crawler:** Inhalte automatisch aus Seiten extrahieren
- **Interne Systeme:** strukturierte Daten ohne separates JSON-Format
- **Semantic Web / schema.org:** zusätzliche Bedeutung direkt im HTML

Kurz gesagt: Microformats/Microdata ergänzen Semantik dort, wo reine Textstruktur nicht reicht.

schema.org: gemeinsames Vokabular für strukturierte Daten

Wofür?

schema.org liefert standardisierte Typen und Eigenschaften, damit Suchmaschinen und andere Systeme wissen, ob ein HTML-Abschnitt eine Person, ein Product, ein Event oder eine JobPosting beschreibt.

Wichtige Idee

- **Typ** sagt, was ein Ding ist
- **Properties** beschreiben Details
- dieselben Typen können in Microdata oder JSON-LD genutzt werden

Typenhierarchie

```
Thing
+-- Person
+-- Organization
|   +-- LocalBusiness
+-- CreativeWork
|   +-- Article
|   +-- Course
+-- Event
+-- Product
```

Beispiele für Properties

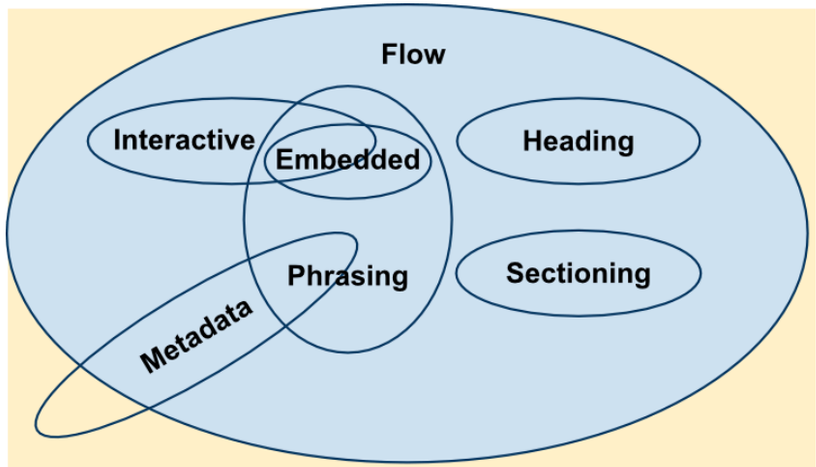
- Person: name, jobTitle, affiliation
- Product: name, offers, brand
- Event: name, startDate, location

```
|   +-- Vehicle
+-- Place
    +-- LocalBusiness
```

HTML Content Categories

HTML-Elemente gehören zu formalen Kategorien. Diese Kategorien sagen nicht nur, **was** ein Element bedeutet, sondern auch **wo** es im Dokument eingesetzt werden soll.

Kategorie	Idee	Typische Elemente
Flow Content	allgemeine Bausteine im `<body>`	`<div>`, `<p>`, `<section>`, `<ul>`
Phrasing Content	Inhalt im laufenden Text	`<span>`, `<a>`, `<em>`, `<img>`
Heading Content	Überschriftenstruktur	`<h1>` bis `<h6>`
Sectioning Content	neue inhaltliche Abschnitte	`<article>`, `<section>`, `<nav>`, `<aside>`
Embedded Content	eingebettete externe Inhalte	`<img>`, `<video>`, `<iframe>`
Interactive Content	direkt bedienbare Elemente	`<a>`, `<button>`, `<input>`
Metadata Content	Dokument-Metadaten im `<head>`	`<title>`, `<meta>`, `<link>`



Warum diese Kategorien wichtig sind

Wesentliche Regel

Phrasing Content darf in laufendem Text stehen.
Flow Content ist allgemeiner und darf viele Blockelemente enthalten.
Deshalb ist ein `<span>` in einem `<p>` normal, ein `<div>` in einem `<p>` aber **invalides HTML**.

```
<p>Gültig: <span>Inline-Inhalt</span></p>
<p>Ungültig: <div>Block-Inhalt</div></p>
```

Praktischer Nutzen

- hilft beim **korrekten Verschachteln** von Elementen
- erklärt Validator-Fehler verständlicher
- verhindert stille Browser-Korrekturen
- macht klar, welche Elemente für Text, Abschnitte oder Interaktion gedacht sind

Browser reparieren ungültiges HTML oft stillschweigend. Genau das macht solche Fehler im Layout später schwer nachvollziehbar.

Formulare: Die Schnittstelle zum Server

Unser Online-Shop braucht ein Registrierungsformular. HTML bietet dafür eingebaute Validierung:

```html
<form action="/api/register" method="POST">
  <label for="email">E-Mail:</label>
  <input type="email" id="email"
         name="email" required>

  <label for="password">Passwort:</label>
  <input type="password" id="password"
         name="password" minlength="8" required>

  <button type="submit">Registrieren</button>
</form>
```

Attribut	Wirkung
`required`	Feld muss ausgefüllt sein
`type="email"`	Browser prüft E-Mail-Format
`minlength`	Mindestlänge des Passworts
`pattern`	Regex-basierte Validierung

> **Diskussionsfrage:** Der Browser validiert clientseitig. Reicht das? Was passiert, wenn jemand das Formular nicht über den Browser, sondern per `curl` oder Fetch API abschickt?

HTML5 als Plattform

HTML5 hat den Browser von einem Dokumentbetrachter zu einer **Laufzeitumgebung** erweitert:

Fähigkeit	API / Element	Typischer Einsatz
Multimedia	`<audio>`, `<video>`	Streaming, Podcasts
Zeichenfläche	`<canvas>`, WebGL	Spiele, Datenvisualisierung
Lokaler Speicher	`localStorage`, `sessionStorage`, IndexedDB	Offline-Daten, Einstellungen
Hintergrundberechnung	Web Workers	CPU-intensive Aufgaben ohne UI-Blockade
Echtzeitkommunikation	WebSockets	Chat, Live-Updates
Geolocation	Geolocation API	Kartenanwendungen
Push-Benachrichtigungen	Notification API + Service Worker	Progressive Web Apps

Architekturentscheidung: Je mehr Fähigkeiten im Browser verfügbar sind, desto mehr Logik **kann** ins Frontend verlagert werden. **Aber sollte sie das?** Ein Online-Shop, der Preise im Client berechnet, riskiert Manipulation. Ein Offline-fähiger Warenkorb dagegen verbessert die User Experience. Die Grenze zwischen Client und Server ist eine bewusste Architekturentscheidung.

Responsive Images

Bilder sollten nicht pauschal in der größten Variante ausgeliefert werden. `img` kann dem Browser mehrere passende Ressourcen anbieten:

Beispiel

```
<img
  src="hero-800.jpg"
  srcset="hero-400.jpg 400w, hero-800.jpg 800w, hero-1600.jpg 1600w"
  sizes="(max-width: 768px) 100vw, 50vw"
  alt="Produktfoto eines Kopfhörers">
```

Semantik

- `src` ist der Fallback
- `srcset` beschreibt das Angebot
- `sizes` beschreibt die erwartete Layout-Breite

Entscheidung

Der Browser kombiniert:

- Layout-Breite aus `sizes`
- verfügbare Dateien aus `srcset`
- Pixeldichte des Geräts

Kernidee

Gleiche Bildinformation, andere Auflösung.
Das ist **Resolution Switching**, kein Ersatz für CSS-Layout.

Details: MDN Responsive images

Responsive Images mit `<picture>`

`<picture>` kommt ins Spiel, wenn nicht nur die Auflösung, sondern **Ausschnitt, Komposition oder Format** wechseln sollen:

Beispiel

```
<picture>
  <source
    media="(max-width: 768px)"
    srcset="hero-mobile.webp">
  <source
    type="image/avif"
    srcset="hero-desktop.avif">
  <img
    src="hero-desktop.jpg"
    alt="Produktfoto eines Kopfhörers">
</picture>
```

> Details: MDN `<picture>`

Semantik

`<source>` beschreibt mögliche Varianten.
Das innere `<img>` ist das tatsächliche Bild-Element.

Wann?

- anderer Bildausschnitt auf Mobile
- anderes Format wie `AVIF` oder `WebP`

Wie

`img[srcset][sizes]` wählt eine Auflösung.
`<picture>` wählt zuerst die passende Bildvariante.

Fallback

Das innere `<img>` bleibt Pflicht: Es ist Fallback, liefert das `alt`-Attribut und wird verwendet, wenn keine `source`-Regel

greift.

Kann HTML das allein? Progressive Enhancement

Bevor man CSS oder JavaScript einsetzt, lohnt die Frage: **Was kann HTML nativ?**

Anforderung	HTML allein?	Erfordert CSS?	Erfordert JS?
Navigation zwischen Seiten	`<a href>` — ja	—	—
Formulardaten an Server senden	`<form action>` — ja	—	—
E-Mail-Format validieren	`<input type="email">` — ja	—	—
Bild mit Beschriftung	`<figure>` + `<figcaption>` — ja	—	—
Dreispaltiges Layout	nein	Grid/Flexbox	—
Hover-Effekt auf Button	nein	`:hover` Pseudo-Klasse	—
Daten vom Server nachladen ohne Seitenreload	nein	nein	`fetch()`
Drag & Drop im Warenkorb	nein	nein	Event Handling

Progressive Enhancement Beginne mit funktionierendem HTML. Füge CSS für Darstellung hinzu. Ergänze JavaScript nur wo nötig. Jede Schicht **erweitert**, statt die vorherige zu ersetzen.

Takeaway

HTML definiert die Struktur und Semantik von Web-Inhalten. Der Unterschied zwischen `<div>`-Suppe und semantischem HTML ist nicht kosmetisch — er bestimmt Barrierefreiheit, Auffindbarkeit und Wartbarkeit. Mit HTML5-APIs wird der Browser zur Plattform, was die Frage aufwirft, wie viel Logik ins Frontend gehört. HTML ist deshalb keine reine Markup-Sprache, sondern die Grundlage der Frontend-Architektur.

JavaScript — Die Sprache des Webs

Leitfrage: JavaScript ist die einzige Programmiersprache, die nativ in jedem Browser läuft. Was muss man über sie wissen, um Web-Systeme architektonisch zu verstehen?

JavaScript in 60 Sekunden

Was ist JavaScript?

- die **native Programmiersprache des Browsers**
- wird meist über `<script>` oder Bundles ausgeliefert
- arbeitet ereignisgesteuert und kann auf DOM und Web APIs zugreifen

Historischer Kontext

- 1995 bei Netscape entstanden
- entworfen von **Brendan Eich**
- trotz des Namens **nicht** mit Java verwandt

Welche Art Sprache ist das?

- dynamisch typisiert
- multiparadigmatisch
- JIT-kompiliert in modernen Browsern
- für kurze Interaktionen und lange Anwendungen gleichermaßen nutzbar

Rolle im Frontend

JavaScript macht statisches HTML zu einer **interaktiven Anwendung**.

Kurzes Beispiel: Verhalten im Browser

```html
<!doctype html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>Warenkorb-Beispiel</title>
  </head>
  <body>
    <button id="buy">In den Warenkorb</button>
    <p id="status">Noch nichts passiert.</p>

    <script>
      const button = document.querySelector('#buy');
      const status = document.querySelector('#status');

      button.addEventListener('click', () => {
        status.textContent = 'Produkt hinzugefügt.';
      });
```

Wann läuft das Skript? Beim Parsen des HTML stoppt der Browser an `<script>`, führt den Inhalt **synchron** aus und parst danach weiter. Weil der Tag *am Ende des `<body>`* steht, existieren `#buy` und `#status` bereits — `querySelector` findet sie. `addEventListener` registriert nur den Callback; der Handler-Code läuft erst, wenn der Browser später ein `click`-Event dispatcht.

Position / Attribut	Wann läuft das Skript?	Risiko
`<script>` am Ende von `<body>` *(hier)*	nach Parsen aller Elemente davor	—
`<script>` im `<head>`, ohne Attribut	sofort, **vor** Body-Aufbau	DOM-Elemente noch nicht da → `querySelector` liefert `null`
`<script defer src=⋯>` im `<head>`	nach Fertigparsen, vor `DOMContentLoaded`	empfohlen für externe Skripte
`<script async src=⋯>` im `<head>`	sobald geladen — Reihenfolge nicht garantiert	nur für unabhängige Skripte (Analytics o. ä.)

> HTML liefert die Elemente. Das `<script>` registriert das Verhalten. Erst der Klick löst die DOM-Änderung aus.

Alternative Code-Auslieferung im Browser

JavaScript ist die **native Standardsprache** des Browsers. In der Praxis tauchen daneben oft noch **TypeScript** und **WebAssembly (WASM)** auf.

Modell	Typische Herkunft	Stärken	Grenzen
JavaScript	direkt im Browser ausgeführt	DOM, Events, schnelle UI-Entwicklung, riesiges Ökosystem	dynamische Typisierung kann große Codebasen schwieriger machen
TypeScript	Erweiterung von JavaScript mit Typen	bessere Wartbarkeit, Tooling, Fehler früher sichtbar	läuft nicht direkt im Browser, muss zu JavaScript kompiliert werden
WebAssembly (WASM)	kompiliert aus Rust, C/C++, Go u. a.	hohe Ausführungsgeschwindigkeit, Portierung bestehender Bibliotheken, numerische/mediale Workloads	kein Ersatz für DOM-APIs, meist JavaScript als Host nötig

Einordnung

- **JavaScript bleibt die Orchestrierungsschicht** für UI, Events und Browser-APIs.
- **TypeScript** ist keine Konkurrenz, sondern eine Entwicklungsvariante von JavaScript.
- **WASM ergänzt JavaScript** dort, wo Rechenleistung oder bestehender Native-Code wichtiger sind als DOM-Nähe.
- Typische Beispiele: Bild- und Videobearbeitung, Audio, CAD, Spiele, In-Browser-Datenverarbeitung.

Der DOM: HTML als programmierbare Struktur

Der Browser parst HTML und erzeugt den **Document Object Model (DOM)** — einen Baum, den JavaScript lesen und verändern kann:

DOM nach HTML-Parsing

```
document
└── html
    ├── head
    └── body
        └── article.product
            ├── h2  "Funkkopfhörer"
            ├── p.price  "€ 79,99"
            └── button
                ├── "In den Warenkorb"
                └── span.cart-badge  "0"
```

JavaScript verändert den Baum

```
// Preis aus dem DOM lesen
const price = document
  .querySelector('.price')
  .textContent;

// Badge aktualisieren
const badge = document
  .querySelector('.cart-badge');
badge.textContent = '3';

// Neues Element erzeugen
const msg = document.createElement('p');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);
```

Kernidee: Der DOM ist die **Brücke zwischen HTML und JavaScript**. Jede Änderung am DOM wird sofort im Browser sichtbar.

Der DOM: HTML als programmierbare Struktur

Der Browser parst HTML und erzeugt den **Document Object Model (DOM)** — einen Baum, den JavaScript lesen und verändern kann:

DOM nach HTML-Parsing

```
document
└── html
    ├── head
    └── body
        └── article.product
            ├── h2  "Funkkopfhörer"
            ├── p.price  "€ 79,99"
            └── button
                ├── "In den Warenkorb"
                └── span.cart-badge  "0"
```

DOM nach Skript-Ausführung

```
document
└── html
    ├── head
    └── body
        ├── article.product
        │   ├── h2  "Funkkopfhörer"
        │   ├── p.price  "€ 79,99"
        │   └── button
        │       ├── "In den Warenkorb"
        │       └── span.cart-badge  "3"   ← geändert
        └── p  "Hinzugefügt!"              ← neu
```

JavaScript verändert den Baum

```
// Preis aus dem DOM lesen
const price = document
  .querySelector('.price')
  .textContent;

// Badge aktualisieren
const badge = document
  .querySelector('.cart-badge');
badge.textContent = '3';

// Neues Element erzeugen
const msg = document.createElement('p');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);
```

Kernidee: Der DOM ist die **Brücke zwischen HTML und JavaScript**. Jede Änderung am DOM wird sofort im Browser sichtbar.

JavaScript als Brücke zwischen HTML und CSS

JavaScript orchestriert Verhalten: Es liest HTML-Zustand, verändert den DOM und kann CSS über Klassen oder Custom Properties auslösen.

Drei Strukturen

- **HTML → DOM:** Struktur der Seite
- **CSS → CSSOM:** berechnete Darstellung
- **JavaScript:** liest und ändert beide zur Laufzeit

```
const button = document.querySelector('#buy');
const badge = document.querySelector('.cart-badge');

button.classList.add('loading');  // CSS reagiert
badge.textContent = '3';          // DOM ändert sich

document.documentElement
  .style.setProperty('--cart-count', '3');
```

Abhängigkeit

```
HTML  ← JavaScript → CSS
DOM      Verhalten    CSSOM
```

Konsequenz

CSS übernimmt Darstellung und Animation.
JavaScript setzt nur den Zustand:

HTML und CSS funktionieren in vielen Fällen ohne JavaScript. JavaScript erweitert sie, wenn neuer Zustand oder Serverkommunikation nötig wird.

- Klasse hinzufügen
- Attribut ändern
- DOM-Knoten erzeugen

Events: Auf Nutzeraktionen reagieren

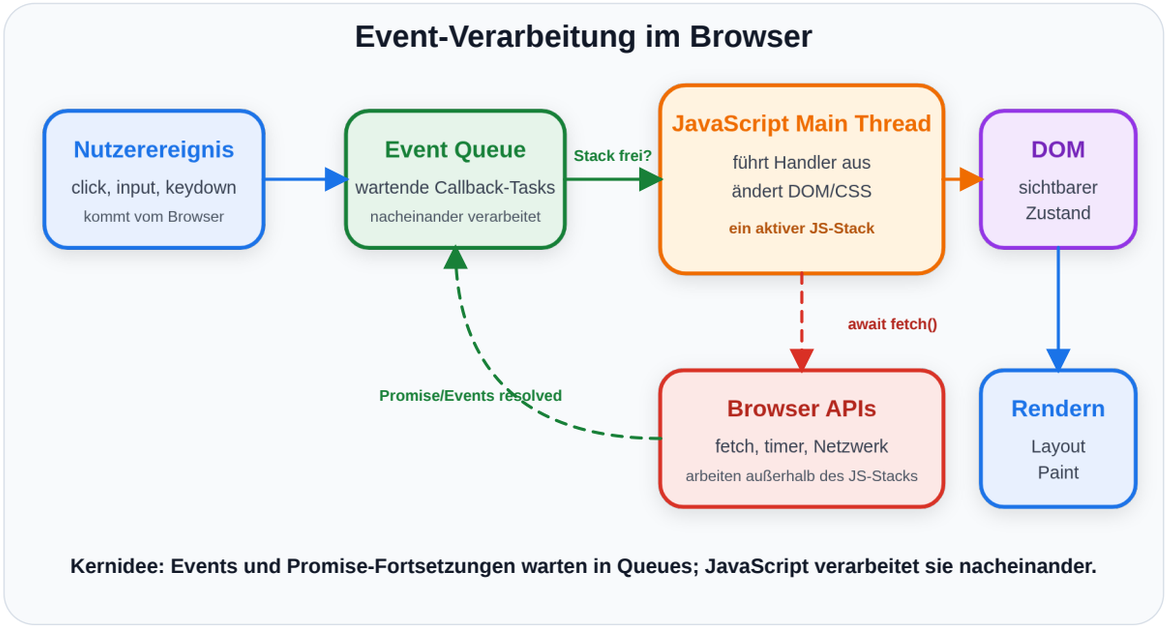
JavaScript arbeitet **ereignisgesteuert**: Code wird ausgeführt, wenn etwas passiert. In unserem Online-Shop:

```
const button = document.querySelector('#add-to-cart');

button.addEventListener('click', async (event) => {
  button.textContent = 'Wird hinzugefügt...';
  button.disabled = true;

  const response = await fetch('/api/cart', {
    method: 'POST',
    body: JSON.stringify({ productId: 42, quantity: 1 })
  });

  if (response.status === 201) {
    button.textContent = '✓ Im Warenkorb';
    updateCartBadge();
  }
});
```



Diskussionsfrage: Der Button wird sofort deaktiviert und der Text geändert, **bevor** die Serverantwort eintrifft. Warum macht man das? Was passiert, wenn die Netzwerkverbindung abbricht?

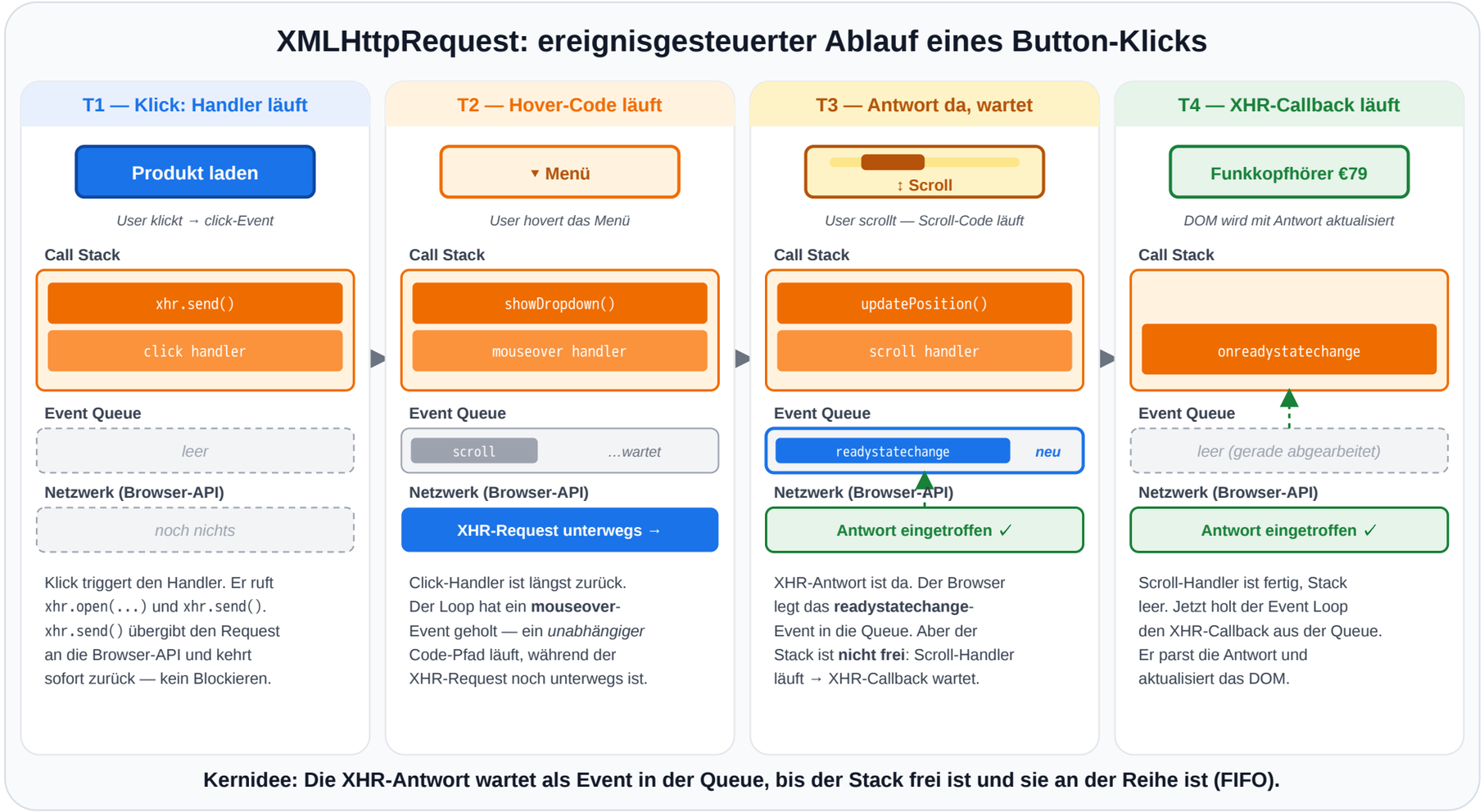
XMLHttpRequest: Der Vorgänger

Vor der Fetch API war `XMLHttpRequest` (XHR) die einzige Möglichkeit, Daten asynchron vom Server nachzuladen — die technische Grundlage für **AJAX** (Asynchronous JavaScript and XML) ab ca. 2005.

```
// GET-Request mit XMLHttpRequest
const xhr = new XMLHttpRequest();
xhr.open('GET', '/api/products/42');
xhr.onreadystatechange = function () {
  if (xhr.readyState === 4) {              // 4 = DONE
    if (xhr.status === 200) {
      const product = JSON.parse(xhr.responseText);
      renderProduct(product);
    } else {
      showError('Fehler: ' + xhr.status);
    }
  }
};
xhr.onerror = function () { showError('Netzwerkfehler'); };
xhr.send();
```

Schwäche von XHR	Konsequenz
Callback-basiert, kein Promise	Verschachtelung bei mehreren Requests („Callback Hell")
Komplexe State-Maschine (`readyState` 0–4)	Boilerplate, fehleranfällig
Konfiguration verteilt auf Properties + Methoden	Kein flüssiger, deklarativer Aufruf
JSON manuell mit `JSON.parse` parsen	Mehr Schritte, mehr Fehlerquellen

XHR im Ablauf: Antwort kommt erst beim Event — dazwischen läuft anderer Code



JavaScript-Ökosystem: Überblick

Bereich	Technologie	Rolle
Browser-Frameworks	React, Vue.js, Angular, Svelte	Deklarative, komponentenbasierte UI
Server-Runtime	Node.js, Deno, Bun	Dasselbe Event-Loop-Modell serverseitig
Full-Stack Frameworks	Next.js, Nuxt.js, SvelteKit	Server-Rendering + Client-Interaktion
TypeScript	Typisierte Obermenge von JS	Statische Typprüfung für große Codebases
Build-Tools	Vite, webpack, esbuild	Bündelung, Transpilation, Optimierung

Die Wahl des Frameworks ist eine **Architekturentscheidung** — sie bestimmt, wie Zustand verwaltet, Code organisiert und die Anwendung deployt wird.

Takeaway

JavaScript ist die **Verhaltensschicht** des Webs und zugleich der Orchestrator: Es ist die einzige Technologie, die sowohl DOM (HTML) als auch CSSOM (CSS) verändern kann. Das Ausführungsmodell — single-threaded mit Event Loop — ermöglicht responsive UIs trotz Netzwerk-Latenz. Die Fetch API verbindet Frontend mit Backend. Für Web Engineering sind drei Konzepte entscheidend: das **Event-Loop-Modell**, die **Abhängigkeitsrichtung** (JS erweitert HTML/CSS, nicht umgekehrt) und die Trennung zwischen **Sprachkern** und **Browser-APIs**.

CSS — Layout und Responsive Design

Leitfrage: Warum braucht das Web eine eigene Sprache für Darstellung — und wo endet die Verantwortung von CSS, wo beginnt die von JavaScript?

CSS in 60 Sekunden

CSS steht für Cascading Style Sheets und beschreibt, **wie HTML dargestellt wird**: Farben, Abstände, Schrift, Layout, Zustände und Responsivität.

Regel

```css
p.highlight {
  color: orange;
  font-weight: 600;
}
```

Semantik

- **Selektor**: `p.highlight` wählt Elemente aus
- **Deklaration**: `color: orange` setzt Stil
- **Regelblock**: alles zwischen `{ ... }`

Browser entscheidet

- Kaskade: Welche Regel gilt?
- Spezifität: Welche Regel ist stärker?
- Vererbung: Was wandert zu Kindern?

Kernidee

CSS ist **deklarativ**: Man beschreibt Regeln und Constraints. Der Browser berechnet daraus konkrete Pixel.

Warum CSS als eigene Technologie?

Dasselbe HTML soll in verschiedenen Kontexten unterschiedlich aussehen: Desktop, Mobile, Print, Dark Mode, reduzierte Bewegung.

Ohne CSS

- Darstellung steckt im HTML
- Redesign betrifft viele Dateien
- Layout wird mit Struktur vermischt
- Gerätevarianten werden dupliziert

Mit CSS

- HTML bleibt Struktur und Semantik
- Stylesheet kann viele Seiten steuern
- Medien und Zustände sind abbildbar
- Screenreader lesen die Struktur, nicht das Layout

CSS trennt **Bedeutung** von **Darstellung**. Das ist keine Kosmetik, sondern eine Architekturentscheidung.

Was CSS ohne JavaScript kann

CSS reagiert auf viele Zustände selbst. JavaScript ist erst nötig, wenn neuer Zustand erzeugt oder externe Daten verarbeitet werden.

CSS reicht

- Hover: `button:hover`
- Fokus: `input:focus`
- Formularstatus: `:valid`, `:invalid`
- Geräte-/Theme-Kontext: `@media`
- Übergänge: `transition`, `animation`

JavaScript nötig

- Daten vom Server laden
- DOM-Elemente erzeugen
- Drag & Drop sortieren
- Zustand berechnen oder speichern
- Klassen als Auslöser setzen

Faustregel: CSS beschreibt Darstellung und Übergänge. JavaScript erzeugt oder verändert Zustand.

Welche Farbe hat der Text?

```html
<p id="info" class="highlight">Willkommen im Shop</p>
```

```css
p            { color: blue; }
.highlight   { color: green; }
#info        { color: red; }
p.highlight  { color: orange; }
```

Aufgabe: Der Paragraph matcht alle vier Selektoren. Welche Farbe wird angezeigt — und warum?



Auflösung: Spezifität entscheidet

Selektor	Spezifität (ID, Klasse, Element)	Farbe
`p`	(0, 0, 1)	blue
`.highlight`	(0, 1, 0)	green
`p.highlight`	(0, 1, 1)	orange
`#info`	(1, 0, 0)	**red**

Was bedeutet `(0, 1, 1)`?

Die Werte zählen Selektor-Bestandteile:

- erste Zahl: **IDs**
- zweite Zahl: **Klassen/Pseudo-Klassen**
- dritte Zahl: **Elemente**

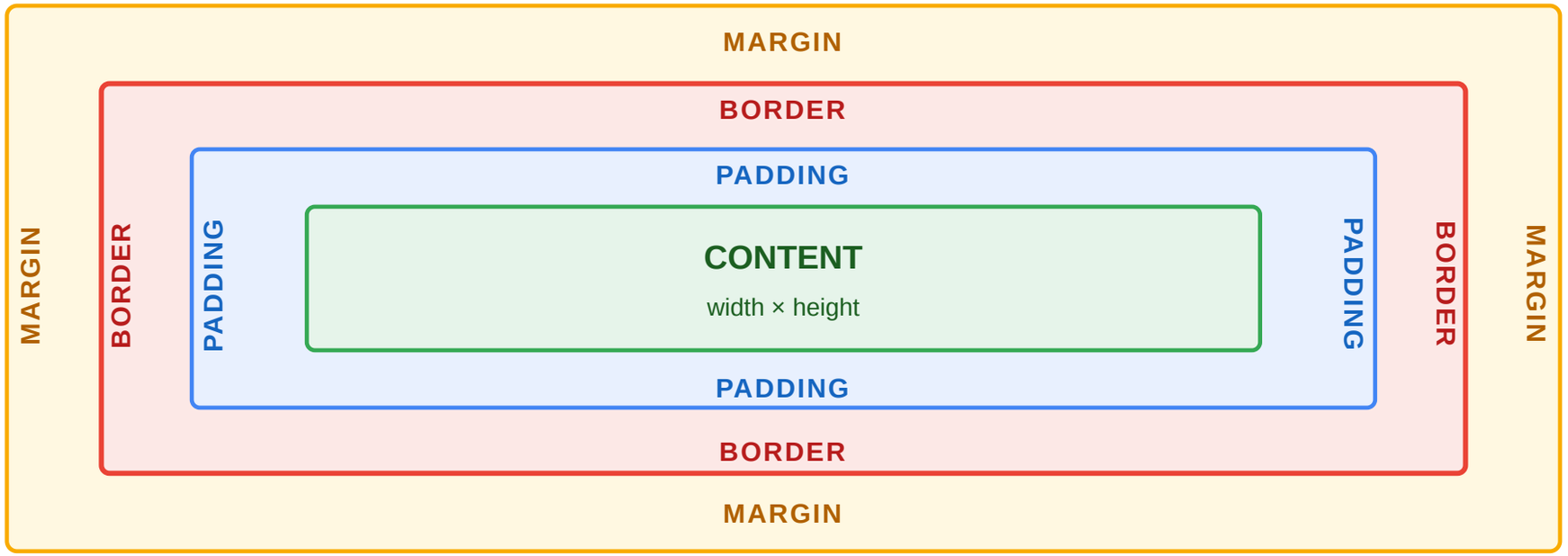
Beispiel: `p.highlight` = 0 IDs, 1 Klasse, 1 Element → `(0, 1, 1)`.
Verglichen wird von links nach rechts.

ID schlägt Klasse schlägt Element. In großen Projekten führen Spezifitätskonflikte zu unerwartetem Verhalten — deshalb setzen moderne Projekte auf Konventionen (BEM) oder komponentenbezogenes CSS (Scoped Styles).

CSS-Layouts: Das Box-Modell

Wie werden HTML-Elemente überhaupt als Layout dargestellt?
CSS betrachtet jedes Element als Box: **Content** → **Padding** → **Border** → **Margin**.

Die häufigste Fehlerquelle: `width: 200px` bedeutet standardmäßig nur den Content — Padding und Border kommen **obendrauf**. Erst `box-sizing: border-box` macht `width` zum Gesamtmaß.



Layoutmodelle im Überblick

Nicht jedes CSS-Layoutproblem ist dasselbe. Die wichtigsten Modelle beantworten unterschiedliche Fragen:

Normal Flow

Default-Verhalten von HTML:

- Block-Elemente stapeln sich untereinander
- Inline-Elemente fließen im Text
- Ausgangspunkt für jedes Layout

Flexbox

Verteilt Elemente entlang **einer Achse**:

- horizontal oder vertikal
- mit Abstand, Reihenfolge und Ausrichtung
- ideal für Reihen, Menüs, Toolbars

Grid

Organisiert Elemente in **Zeilen und Spalten**:

- zweidimensional
- explizite Rasterstruktur
- ideal für Seitenlayout, Dashboards, Galerien

Faustregel

- **Flow**: Was passiert ohne Layoutanweisung?
- **Flexbox**: Wie verteilen wir Inhalt entlang einer Achse?
- **Grid**: Wie definieren wir ein Raster?

Normal Flow: Das Default-Layout des Browsers

Bevor wir Flexbox oder Grid einsetzen, sollten wir verstehen, was HTML **ohne besondere Layoutregeln** tut:

Block-Elemente beginnen typischerweise in einer neuen Zeile und nehmen die verfügbare Breite ein.

Inline-Elemente bleiben im laufenden Text und belegen nur so viel Platz wie ihr Inhalt braucht.

```
<p>Absatz <span>Inline-Text</span> nach dem Wort.</p>
<div>Dieses Block-Element steht darunter.</div>
```

Absatz Inline-Text nach dem Wort.

Dieses Block-Element steht darunter.

```
span {
  display: block;
}

div {
  display: inline;
}
```

Wichtig

HTML-Elemente haben Default-Displays. CSS kann diese Darstellung ändern, ohne die Semantik des Elements zu ändern.

Flow ist deshalb wichtig, weil Flexbox und Grid immer bewusst vom Normalfall abweichen.

Flexbox: Verteilung entlang einer Achse

Flexbox ist das richtige Werkzeug, wenn Elemente in **einer Zeile oder Spalte** organisiert werden sollen:

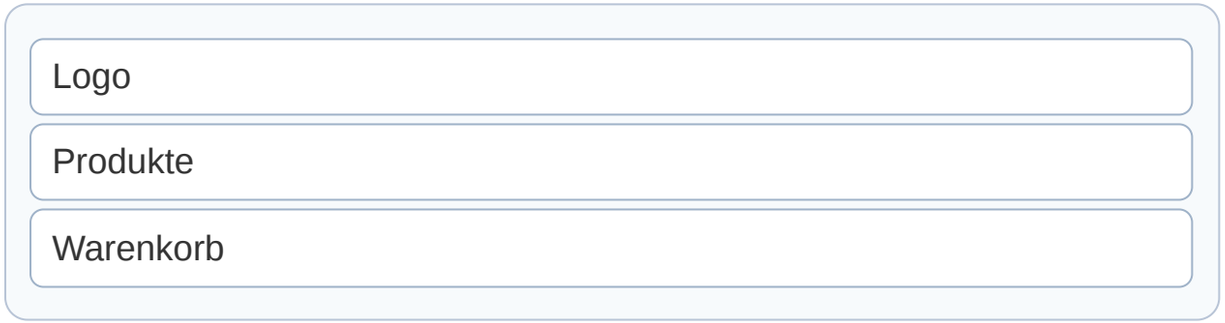
```css
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}
```

`display: flex` steht auf dem **übergeordneten Element** `.nav`. Die direkten Kinder werden dadurch zu Flex-Items.

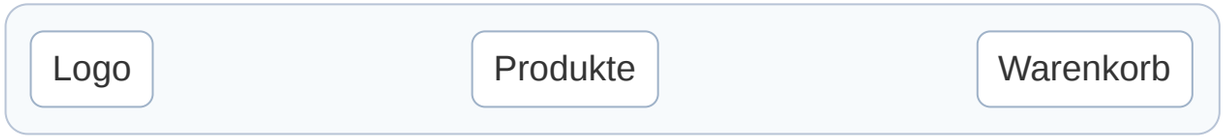
Typische Fragen:

- nebeneinander oder untereinander?
- wie viel Abstand?
- links, mittig oder rechts ausrichten?

Ohne Flex



Mit Flex



Ohne `display: flex` stapeln sich die Block-Elemente im Normal Flow. Mit `display: flex` verteilt der Container seine direkten Kinder entlang einer Achse.

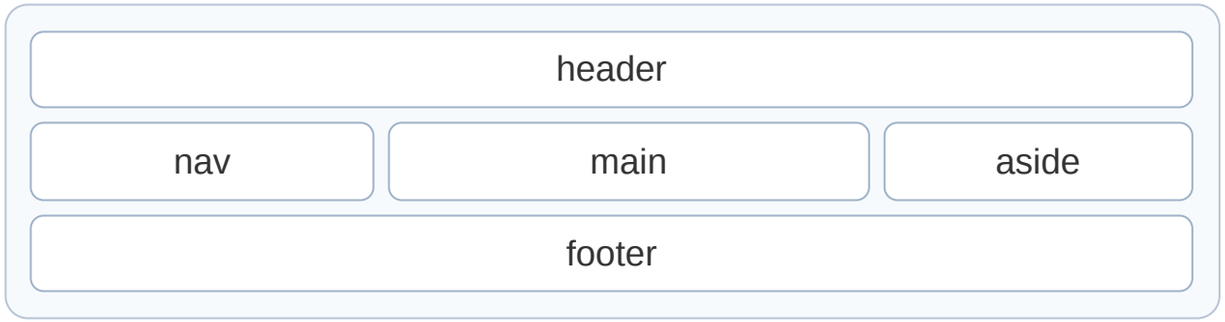
Grid: Ein explizites 2D-Raster

Grid ist sinnvoll, wenn das **Layout selbst** die Struktur vorgibt.
Explizit bedeutet hier: In CSS werden **Spalten**, **Zeilen** und oft auch die **Platzierung der Bereiche** vorgegeben.

```css
.shop-layout {
  display: grid;
  grid-template-columns: 240px 1fr 220px;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header header"
    "nav    main   aside"
    "footer footer footer";
  gap: 1rem;
}
```

`grid-template-columns` definiert die **Spalten**.
`grid-template-rows` definiert die **Zeilen**.
`grid-template-areas` ordnet benannte Bereiche im Raster an.

`1fr` bedeutet: nimm **einen Anteil des verbleibenden freien Platzes**.
`"header header header"` bedeutet: Der Bereich `header` belegt in dieser Zeile **alle drei Spalten**.



Bei Grid entsteht zuerst das Raster. Danach werden die Elemente in dieses Raster eingeordnet.

Responsives Grid ohne starre Breakpoints

Responsive Grid ist **weiterhin CSS Grid**. Jedoch ist die Anzahl der Spalten nicht fest vorgegeben:

Fixes Grid

```css
.product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
```

Immer drei Spalten — auch wenn der Platz knapp wird.

Responsives Grid

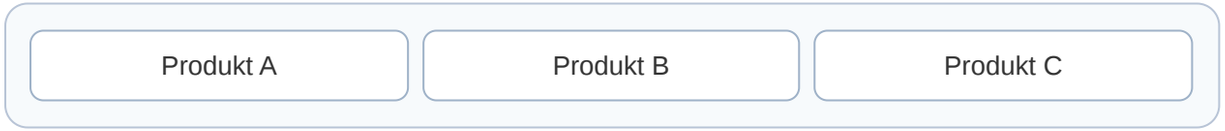
```css
.product-grid {
  display: grid;
  gap: 1rem;
  grid-template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
}
```

So viele Spalten wie passen — sonst Umbruch.

> `minmax(220px, 1fr)` bedeutet:
>
> - nie schmaler als `220px`
> - sonst flexibel wachsen

> `auto-fit` erzeugt so viele Spalten wie möglich.
> `minmax()` begrenzt jede Spalte zwischen Minimum und flexiblem Wachstum.

Breiter Container



Schmaler Container



> Der Browser berechnet die Spaltenanzahl aus verfügbarem Platz und Mindestbreite. Keine eigene Media

Beispiel: Produktdetailseite mit Bild und Text

Beispiel-Code

```css
.product {
  display: grid;
  grid-template-areas: "image info";
  grid-template-columns: 1fr 1.2fr;
  gap: 2rem;
}

@media (max-width: 700px) {
  .product {
    grid-template-areas: "image" "info";
    grid-template-columns: 1fr;
  }
}
```

Situation: Links steht das Produktbild, rechts Beschreibung und Kaufbutton. Auf dem Smartphone soll das Bild oben und der Text darunter erscheinen.



Warum hier Grid?

- Desktop beschreibt Bereiche als `"image info"`
- Mobile beschreibt Bereiche als `"image"` / `"info"`
- Layout bleibt als Struktur sichtbar
- Abfragbar mittesl CSS Media Queries

Flexbox wäre für eine einfache Zeile ebenfalls möglich. Grid ist hier klarer, weil Desktop- und Mobile-Layout als benannte Bereiche formuliert werden.

Mobile: Bereiche untereinander




```
"image" / "info"
```

Welche Wahl ist hier besser?

Flexbox passt, wenn

- nur eine Hauptachse wichtig ist
- die Elemente einfach verteilt werden
- Reihenfolge und Struktur weitgehend linear bleiben

Grid passt, wenn

- Zeilen und Spalten gemeinsam wichtig sind
- Bereiche explizit benannt werden sollen
- responsives Umschalten strukturell klar bleiben soll

Faustregel: Flexbox, wenn der Inhalt verteilt wird. Grid, wenn das Layout als Raster vorgegeben wird.

Takeaway

CSS existiert als eigene Sprache, weil dasselbe HTML in **verschiedenen Kontexten** verschieden dargestellt werden muss — Screen, Print, Mobile, Dark Mode. CSS ist **deklarativ**: Man beschreibt Constraints, der Browser berechnet Pixel. Spezifität bestimmt, welche Regel gewinnt. Flexbox und Grid lösen unterschiedliche Verteilungsprobleme (Inhalt verteilen vs. Raster vorgeben). Responsive Design mit Mobile-First ist Voraussetzung. Die wichtigste Grenzfrage: **Kann CSS das allein** (Hover, Transition, Media Query), oder braucht es JavaScript? Nur wenn neuer Zustand erzeugt werden muss, ist JS die richtige Schicht.

Web APIs — die Schnittstellen des Browsers

Leitfrage: Was sind Web APIs — und warum gehören sie nicht zur JavaScript-Sprache selbst, sondern zum Browser?

Web APIs: Definition und Überblick

Web APIs sind Browser-Schnittstellen, die JavaScript zusätzliche Fähigkeiten geben: den DOM verändern, Netzwerke ansprechen, Daten speichern, Gerätefunktionen nutzen oder Medien steuern.

Typische Browser-APIs

- DOM API
- Fetch API
- Web Storage API
- Geolocation API
- Canvas / WebGL
- Web Workers

Warum das wichtig ist

- JavaScript allein beschreibt die Sprache
- Web APIs machen sie im Browser praktisch nutzbar
- viele Web-Funktionen sind erst durch diese APIs möglich

> **Beispiel:** `fetch()` lädt Daten, `localStorage` speichert Zustand, DOM APIs aktualisieren die Oberfläche.

Wofür nutzt man sie?

Aufgabe	Beispiel-API
HTML oder CSS zur Laufzeit ändern	DOM API
Daten vom Server laden	Fetch API
Zustand im Browser speichern	Web Storage API
Standorte abfragen	Geolocation API
Lange Berechnungen auslagern	Web Workers

Kernaussage: Web APIs sind die Schnittstelle zwischen JavaScript und den Fähigkeiten des Browsers.

> **Grenze:** Nicht jede API ist Teil von JavaScript selbst. Viele Funktionen kommen vom Browser, nicht von der Sprache.

Fetch API: Die Client-Seite von HTTP

Die Fetch API ist die Brücke zwischen Frontend (Vorlesung 03) und Protokoll (Vorlesung 04):

```
// GET: Produktliste laden
const response = await fetch('/api/products?category=electronics&sort=-price');
const products = await response.json();

// POST: Bestellung aufgeben
const result = await fetch('/api/orders', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ productId: 42, quantity: 1 })
});
if (result.status === 201) {
  const order = await result.json();
  window.location.href = `/orders/${order.id}`;  // Weiterleitung
}
```

Fetch-Konzept	HTTP-Gegenstück
`method: 'POST'`	HTTP-Methode — definiert die Absicht
`headers`	HTTP Request Headers — Metadaten
`response.status`	HTTP-Statuscode — Ergebnis der Operation
`response.json()`	Body parsen — die Repräsentation der Ressource

> `fetch()` ist die JavaScript-Abstraktion, aber die Konzepte darunter sind HTTP: Methode, Header, Statuscode und Body.

Was kann beim Fetch schiefgehen?

Aufgabe: Dieser Code funktioniert im Normalfall. Welche Probleme können auftreten — und wie sollte man sie behandeln?

```
const response = await fetch('/api/products/42');
const product = await response.json();
document.querySelector('.name').textContent = product.name;
document.querySelector('.price').textContent = product.price + ' €';
```

```
button.addEventListener('click', async () => {
  try {
    const response = await fetch('/api/products/42');
    if (!response.ok) throw new Error(`HTTP ${response.status}`);

    const product = await response.json();
    renderProduct(product);
  } catch (err) {
    showError('Produkt konnte nicht geladen werden.');
  }
});
```

Das ist ereignisgesteuert **und** Promise-basiert: Der Klick startet den Ablauf, `await` wartet auf die asynchrone Fetch-Fortsetzung.

Was kann beim Fetch schiefgehen?

Aufgabe: Dieser Code funktioniert im Normalfall. Welche Probleme können auftreten — und wie sollte man sie behandeln?

```
const response = await fetch('/api/products/42');
const product = await response.json();
document.querySelector('.name').textContent = product.name;
document.querySelector('.price').textContent = product.price + ' €';
```

```
button.addEventListener('click', async () => {
  try {
    const response = await fetch('/api/products/42');
    if (!response.ok) throw new Error(`HTTP ${response.status}`);

    const product = await response.json();
    renderProduct(product);
  } catch (err) {
    showError('Produkt konnte nicht geladen werden.');
  }
});
```

Problem	Was passiert	Lösung
Server nicht erreichbar	`fetch` wirft Exception	`try/catch` + Fehlermeldung anzeigen
404 Not Found	`response.ok` ist `false`, `.json()` liefert Fehler-Body	Status prüfen vor `.json()`
Langsame Verbindung	UI zeigt alte/keine Daten	Loading-Indicator anzeigen
Response ist kein JSON	`.json()` wirft Exception	`Content-Type` prüfen
DOM-Element existiert nicht	`querySelector` liefert `null` → Fehler	Null-Check oder optionale Verkettung (`?.`)

Robuste Fehlerbehandlung ist keine Kür — sie ist der Unterschied zwischen einem Prototyp und einem Produktivsystem.

Das ist ereignisgesteuert **und** Promise-basiert: Der Klick startet den Ablauf, `await` wartet auf die asynchrone Fetch-Fortsetzung.

Promises und asynchrone Verarbeitung

Ein **Promise** repräsentiert das *zukünftige Ergebnis* einer asynchronen Operation. Es kennt drei Zustände: `pending` → `fulfilled` oder `rejected`.

Klassischer Promise-Stil mit Callback-Verkettung

```
fetch('/api/products/42')                  // gibt Promise zurück
  .then(response => {                      // läuft, wenn Header da sind
    if (!response.ok) throw new Error(response.status);
    return response.json();                // gibt wieder ein Promise zurück
  })
  .then(product => renderProduct(product)) // läuft, wenn JSON geparst ist
  .catch(err => showError(err.message));   // fängt Fehler aus jeder Stufe
```

Konzept	Bedeutung
`await`	Pausiert die `async`-Funktion, bis das Promise *settled* ist — blockiert **nicht** den Main Thread
`Promise.all([p1, p2])`	Wartet parallel auf mehrere Promises; scheitert, sobald *eines* rejected
`Promise.allSettled([...])`	Wartet auf alle, gibt Erfolg *und* Fehler zurück
`try / catch` um `await`	Fängt sowohl Netzwerk- als auch Logikfehler — ersetzt `.catch()`

`async / await` — derselbe Ablauf, linearer Code

```
async function loadProduct() {
  try {
    const product = await productApi.fetch(42);
    renderProduct(product);
  } catch (err) {
    showError(err.message);
  }
}
```

> `productApi.fetch(...)` implementiert dieselbe Datenlade-Methode wie die `.then(...)`-Kette: erst `fetch`, dann `response.json()`, dann `return product`. Der UI-Code ruft diese Methode nur noch mit `await` auf.

```
const productApi = {
  async fetch(id) {
    // entspricht: fetch(...).then(response => ...)
    const responsePromise = fetch(`/api/products/${id}`);
    const response = await responsePromise;

    if (!response.ok) throw new Error(response.status);

    // entspricht: response.json().then(product => ...)
    const productPromise = response.json();
    const product = await productPromise;

    return product;
  }
};
```

await im Zeitverlauf: zwei Fetch-Requests



Cookies: kleine Schlüssel-Werte, automatisch zum Server

Cookies sind kleine, vom Browser gespeicherte Schlüssel-Wert-Paare. Im Unterschied zu allen anderen Storage-APIs werden sie bei **jedem** HTTP-Request an die zugehörige Domain *automatisch* mitgeschickt — das macht sie zur klassischen Wahl für Sessions und Authentifizierung.

Setzen — Server (üblich)

```
HTTP/1.1 200 OK
Set-Cookie: session=abc123; Max-Age=3600;
          HttpOnly; Secure; SameSite=Strict
```

Setzen — Client (selten)

```
document.cookie = 'theme=dark; max-age=31536000; SameSite=Lax';
```

Lesen — Client

```
// alle (nicht-HttpOnly) Cookies als ein String
const all = document.cookie;
// → "theme=dark; locale=de-AT"
```

Wichtige Attribute

Attribut	Wirkung
`HttpOnly`	Für JavaScript **unsichtbar** — schützt vor XSS-Diebstahl
`Secure`	Nur über HTTPS übertragen
`SameSite=Strict|Lax|None`	Sendet Cookie nicht (oder nur Top-Level) bei Cross-Site-Requests — schützt vor CSRF
`Max-Age` / `Expires`	Lebensdauer; ohne beides → Session-Cookie
`Domain` / `Path`	Auf welche Hosts/URLs der Cookie geht

Typische Anwendungen

- Session-ID für angemeldete Nutzer
- CSRF-Token, Sprache, Theme
- Tracking / Analytics (Drittanbieter-Cookies)

Faustregel: Auth-Tokens gehören in einen Cookie mit `HttpOnly + Secure + SameSite=Strict`. UI-State

Web Storage: localStorage, sessionStorage, Cookies

Browser-Speicher unterscheidet sich vor allem danach, **wie lange** Daten leben und **wer** sie lesen oder automatisch mitsenden kann.

API	Lebensdauer	Größe	Zugriff
`localStorage`	Persistent (bis manuell gelöscht)	~5–10 MB	Nur Client (JS)
`sessionStorage`	Nur aktuelle Browser-Session	~5–10 MB	Nur Client (JS)
`Cookies`	Konfigurierbar (Session oder Ablaufdatum)	~4 KB	Client + **automatisch an Server**
`IndexedDB`	Persistent	Gigabyte-Bereich	Client (asynchron)

```
// localStorage: einfache Key-Value-Speicherung
localStorage.setItem('cart', JSON.stringify(cartItems));
const cart = JSON.parse(localStorage.getItem('cart') || '[]');

// sessionStorage: nur für die aktuelle Tab-Session
sessionStorage.setItem('wizard-step', '3');

// Cookies: werden bei jedem HTTP-Request mitgesendet
document.cookie = 'theme=dark; max-age=31536000; SameSite=Strict';
```

Wann was?

- **Cookies** für alles, was der **Server wissen muss** (Session-ID, Authentifizierung)
- **localStorage** für Client-State, der **über Sessions** erhalten bleiben soll (Warenkorb, UI-Einstellungen)
- **sessionStorage** für temporären State, der **nur im aktuellen Tab** gilt (Wizard-Fortschritt)
- **IndexedDB** für große Datenmengen oder Offline-Anwendungen

History API: Navigation ohne Seiten-Reload

Single-Page Applications navigieren ohne vollständigen Seiten-Reload. Der Browser-Zurück-Button muss trotzdem funktionieren. Die **History API** macht das möglich:

```
// URL ändern, ohne neu zu laden
history.pushState({ page: 'products' }, '', '/products');

// Neue Inhalte fetch'en und rendern
const data = await fetch('/api/products').then(r => r.json());
renderProducts(data);

// Zurück-Button abfangen
window.addEventListener('popstate', (event) => {
  if (event.state?.page === 'products') renderProducts(/* ... */);
});
```

Methode / Event	Zweck
`history.pushState(state, '', url)`	URL ändern, Eintrag zum Verlauf hinzufügen
`history.replaceState(...)`	URL ändern, **ohne** neuen Eintrag
`popstate` Event	Browser-Zurück-/Vorwärts-Button

> Client-side Routing muss immer zwei Wege behandeln: aktive Navigation per `pushState` und passive Navigation über den Zurück-/Vorwärts-Button.

Web Workers vs Service Workers

Beide laufen in einem **separaten Thread** außerhalb des Haupt-Event-Loops — aber mit unterschiedlichen Zwecken.

Aspekt	Web Worker	Service Worker
Zweck	CPU-intensive Berechnungen auslagern	Netzwerk-Proxy, Offline-Support, Push
Lebensdauer	Gebunden an Tab/Seite	Persistent, auch wenn Tab geschlossen
DOM-Zugriff	Nein	Nein
Fetch-Interception	Nein	Ja — fängt alle Netzwerk-Requests ab
Typischer Einsatz	Bildverarbeitung, Datenanalyse, Kryptographie	Progressive Web Apps, Caching, Push-Benachrichtigungen

```
// Web Worker: teure Berechnung auslagern
const worker = new Worker('compute.js');
worker.postMessage({ data: largeDataset });
worker.onmessage = (e) => displayResult(e.data);

// Service Worker: Fetch abfangen für Offline-Support
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request).then(cached => cached || fetch(event.request))
  );
});
```

> Web Worker entlasten die UI bei Rechenarbeit. Service Worker sitzen zwischen Anwendung und Netzwerk und machen Offline-/Cache-Strategien möglich.

Takeaway

Web APIs sind die **Fähigkeiten**, die der Browser JavaScript zur Verfügung stellt — nicht Teil der Sprache selbst, sondern Erweiterungen. Die wichtigsten Kategorien sind DOM-Manipulation, Netzwerk (Fetch), Speicher (localStorage/Cookies), Hintergrund-Threads (Web/Service Workers) und History. Welche API wofür eingesetzt wird, ist eine Architekturfrage mit Konsequenzen für Sicherheit, Performance und Offline-Fähigkeit.

Moderne Web Frontends

Leitfrage: Nachdem HTML, CSS und JavaScript einzeln geklärt sind: Welche Performance- und Architekturfolgen entstehen aus ihrem Zusammenspiel im Browser?

Warum Frameworks existieren: Das Skalierungsproblem

Unser Warenkorb-Button funktioniert mit `querySelector` und `addEventListener`. Was passiert, wenn die Anwendung wächst?

```
button.addEventListener('click', () => {
  badge.textContent = '3';
});
```

Imperativ: DOM nachführen

- Elemente suchen
- Events registrieren
- DOM manuell aktualisieren
- Zustand an mehreren Stellen synchron halten

Deklarativ: UI aus Zustand

- Zustand beschreiben
- Komponenten rendern
- Framework berechnet DOM-Änderungen
- UI bleibt konsistent zu `state`

Frameworks lösen kein Syntax-Problem, sondern ein **Zustandsmanagement-Problem**: Wie bleibt die UI konsistent, wenn sich Daten ändern?

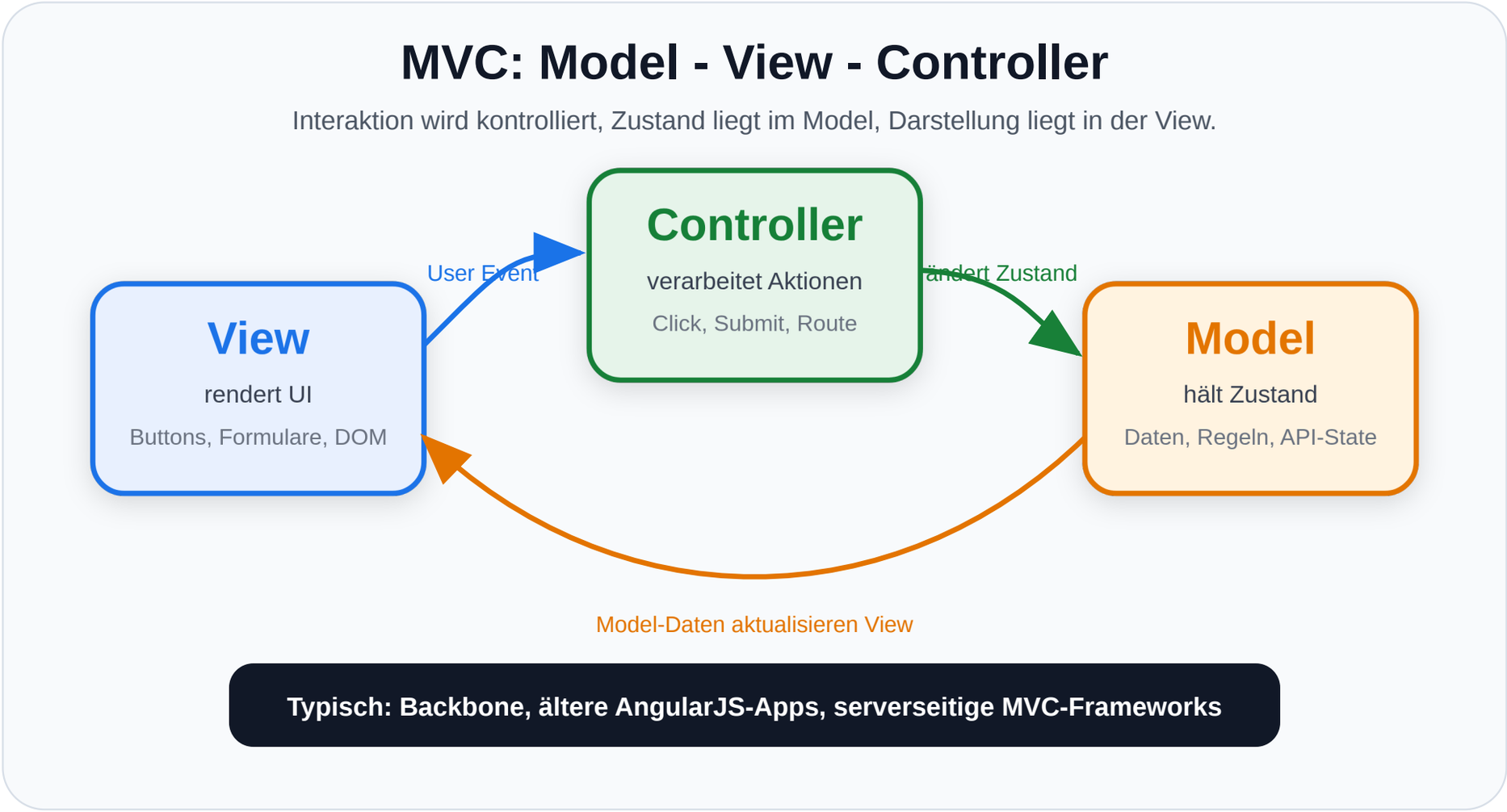
Wie hat sich das Zusammenspiel zwischen HTML/CSS/JS verändert?

Ära	Muster	Zusammenspiel
Klassisch (2000er)	Server rendert HTML, CSS als separate Datei, JS für kleine Effekte	Strikte Trennung: HTML-Datei + CSS-Datei + JS-Datei
jQuery-Ära (2006–2015)	HTML vom Server, JS manipuliert DOM direkt	DOM als zentrale Schnittstelle
Komponentenbasiert (ab 2013)	React/Vue/Angular: HTML + CSS + JS pro Komponente	Trennung nach **Verantwortung**, nicht nach Technologie
Server-First (ab 2020)	Next.js/SvelteKit: Server rendert, Client hydriert	Erst HTML vom Server, dann JavaScript macht es interaktiv

Client-hydriert bedeutet: Der Server liefert bereits fertiges HTML an den Browser. Danach lädt JavaScript nach und **bindet Verhalten an dieses vorhandene HTML** (Event-Listener werden registriert, Komponenten bekommen ihren interaktiven Zustand, die Seite wird nutzbar, ohne dass der Browser alles neu rendern muss).

> Welche Architekturen gibt es nun zur Auftrennung von Darstellung (View), Daten (Model) und Kontrolllogik (Control).

MVC: Model - View - Controller



MVC trennt Darstellung, Zustand und Eingabeverarbeitung. Die View zeigt Daten, der Controller verarbeitet Aktionen, das Model hält fachlichen Zustand.

MVC am Mini-Beispiel

View

```
<input id="name">
<button id="save">Speichern</button>
<p id="output"></p>
```

Model

```
const model = { name: '' };
```

Controller

```
document.querySelector('#save')
  .addEventListener('click', () => {
    model.name = document.querySelector('#name').value;
    render();
  });

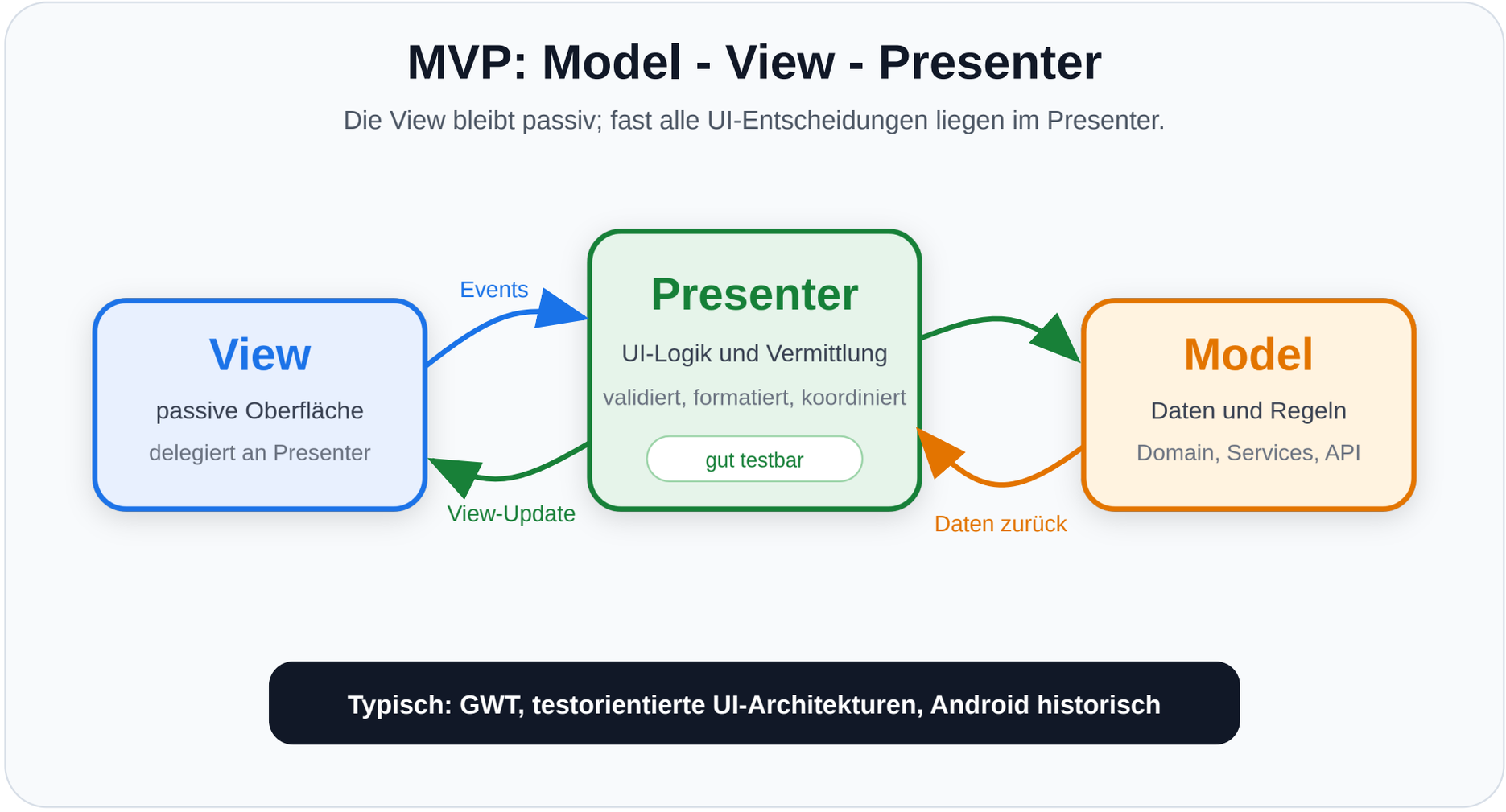
function render() {
  document.querySelector('#output').textContent =
    `Hallo ${model.name}`;
}
```

Vorteil: Sehr direkt: Event → Controller → Model → View.

Nachteil: Schwerer isoliert testbar: Controller kennen DOM-Details, und View-Code kann eigene Logik enthalten.

MVC: Der Controller reagiert auf die Eingabe, ändert das Model und stößt das Rendering der View an.

MVP: Model - View - Presenter



MVP macht die View passiv. Der Presenter enthält die UI-Logik und spricht die View über ein Interface an.

MVP am Mini-Beispiel

View: Oberfläche + Interface

```html
<input id="name">
<button id="save">Speichern</button>
<p id="output"></p>
```

```
const view = {
  getName: () =>
    document.querySelector('#name').value,
  showGreeting: (text) =>
    document.querySelector('#output').textContent = text
};
```

Model + Presenter

```
const model = { name: '' };

const presenter = {
  save() {
    model.name = view.getName();
    view.showGreeting(`Hallo ${model.name}`);
  }
};

document.querySelector('#save')
  .addEventListener('click', () => presenter.save());
```

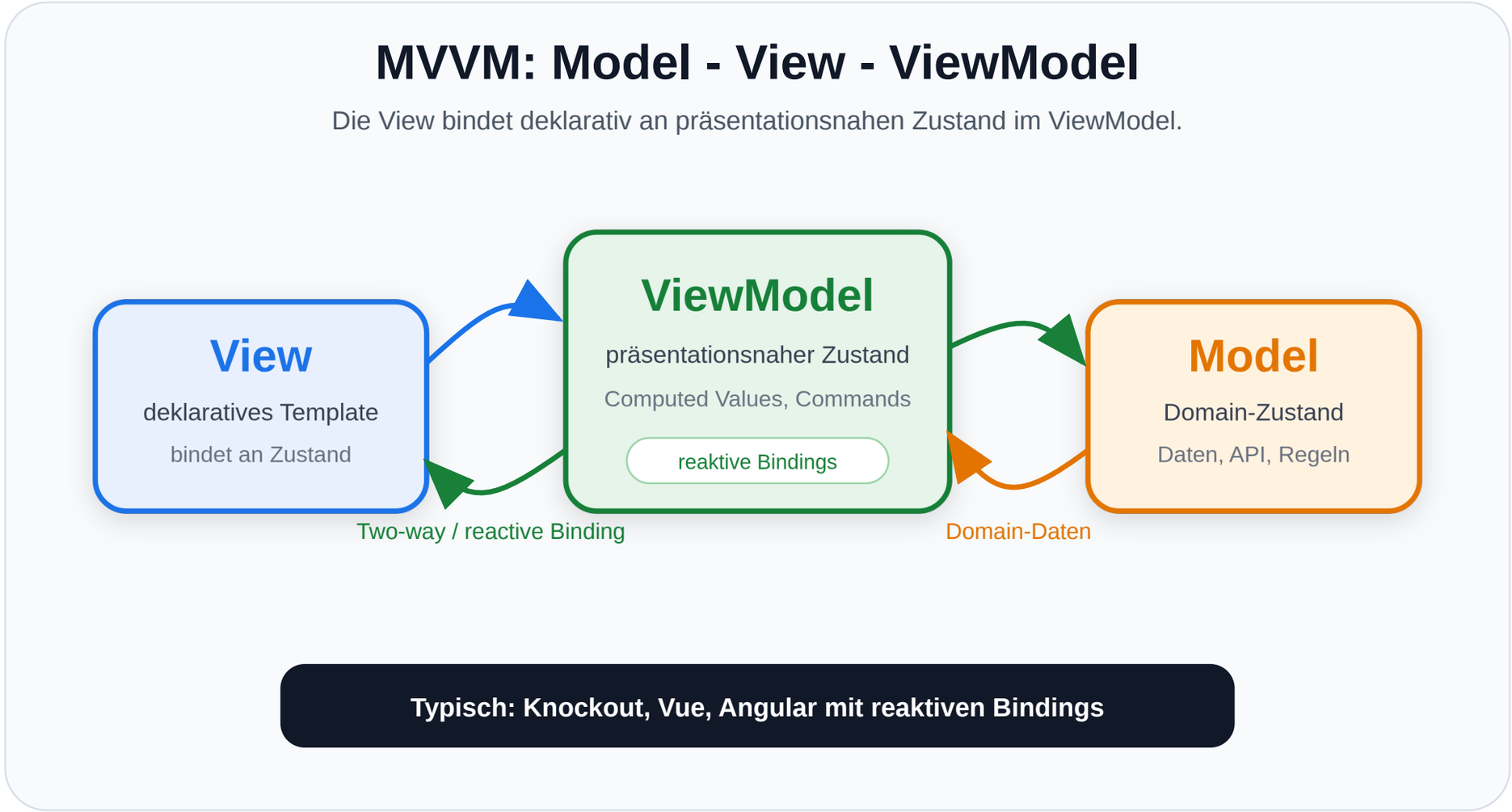
Vorteil: UI-Logik ist gut testbar, weil der Presenter ohne echte View geprüft werden kann.

Nachteil: Mehr Boilerplate: View-Interfaces, Presenter und Weiterleitungen müssen gepflegt werden.

MVP: Die View bleibt passiv. Der Presenter liest aus der View, ändert das Model und sagt der View, was sie

anzeigen soll.

MVVM: Model - View - ViewModel



MVVM verbindet die View deklarativ mit einem ViewModel. Änderungen am Zustand führen über Bindings automatisch zu UI-Aktualisierungen.

MVVM am Mini-Beispiel

View

```
<input data-bind="name">
<button data-action="save">Speichern</button>
<p data-bind="greeting"></p>
```

Model

```
const model = { name: '' };
```

ViewModel

```
const viewModel = {
  name: '',
  get greeting() {
    return `Hallo ${this.name}`;
  },
  save() {
    model.name = this.name;
  }
};
```

`data-bind` steht hier für Framework-Bindings wie in Knockout, Vue oder Angular.

Vorteil: Deklarative Bindings reduzieren manuelle DOM-Aktualisierung.

Nachteil: Implizite Bindings können Debugging erschweren: Änderungen passieren automatisch und nicht immer sichtbar im Codepfad.

MVVM: Die View bindet deklarativ an das ViewModel. Der sichtbare Text ergibt sich aus Zustand und abgeleiteten Werten.

Architekturfamilien moderner Frontends

Familie	Kernidee	Typische Verantwortung	Beispiele
MVC	Eingaben steuern einen Controller, der Model und View koppelt	View rendert, Controller verarbeitet Aktionen	Backbone, ältere AngularJS-Apps, serverseitige MVC-Frameworks
MVP	Presenter vermittelt explizit zwischen passiver View und Model	Presenter enthält fast die gesamte UI-Logik	GWT, viele testorientierte UI-Architekturen, Android historisch
MVVM	View bindet deklarativ an ein ViewModel	ViewModel hält präsentationsnahen Zustand	Vue, Knockout, Angular mit deutlichen MVVM-Elementen

MVC

- gut für klare Rollentrennung
- Controller wird in großen UIs schnell überladen

MVP

- stark testbar, weil die View passiv bleibt
- oft mehr Boilerplate durch Presenter-Schnittstellen

MVVM

- passt gut zu deklarativen Bindings
- heute das prägendste Denkmodell vieler Browser-Frameworks

Moderne Frameworks: Familien als Bezugspunkt

Einordnung von gängigen Frameworks

- **Backbone**: nah an MVC
- **Knockout**: klassisches MVVM
- **Vue**: oft MVVM-artig mit Komponenten
- **Angular**: komponentenbasiert, mit MVVM-Elementen
- **React**: Komponentenmodell mit unidirektionalem Datenfluss

Was daraus folgt

- Frameworks sind selten „reines MVC“ oder „reines MVVM“
- sie übernehmen Ideen aus mehreren Familien
- entscheidend ist der Datenfluss: Wer hält Zustand, wer rendert, wer reagiert?
- Komponenten kapseln heute oft View + Verhalten + lokalen Zustand

Wrap-Up

- **HTML** definiert Struktur und Semantik — die Grundlage für Barrierefreiheit, SEO und maschinelle Verarbeitung.
- **JavaScript** macht den Browser programmierbar — über DOM, Events und asynchrone Kommunikation mit dem Server.
- **CSS** trennt Darstellung von Struktur — Flexbox und Grid lösen das Layout-Problem, Responsive Design das Geräteproblem.
- Die drei Technologien sind **eng verzahnt**: Der Browser verarbeitet sie in einer Pipeline (Parse → Render Tree → Layout → Paint), die JavaScript jederzeit beeinflussen kann.

Die nächste Vorlesung vertieft das **Protokoll** (HTTP) und das **API-Design** (REST), das die Brücke zwischen Frontend und Backend bildet.